SYSTEM ARCHITECTURE AND DATABASE DIAGRAMS

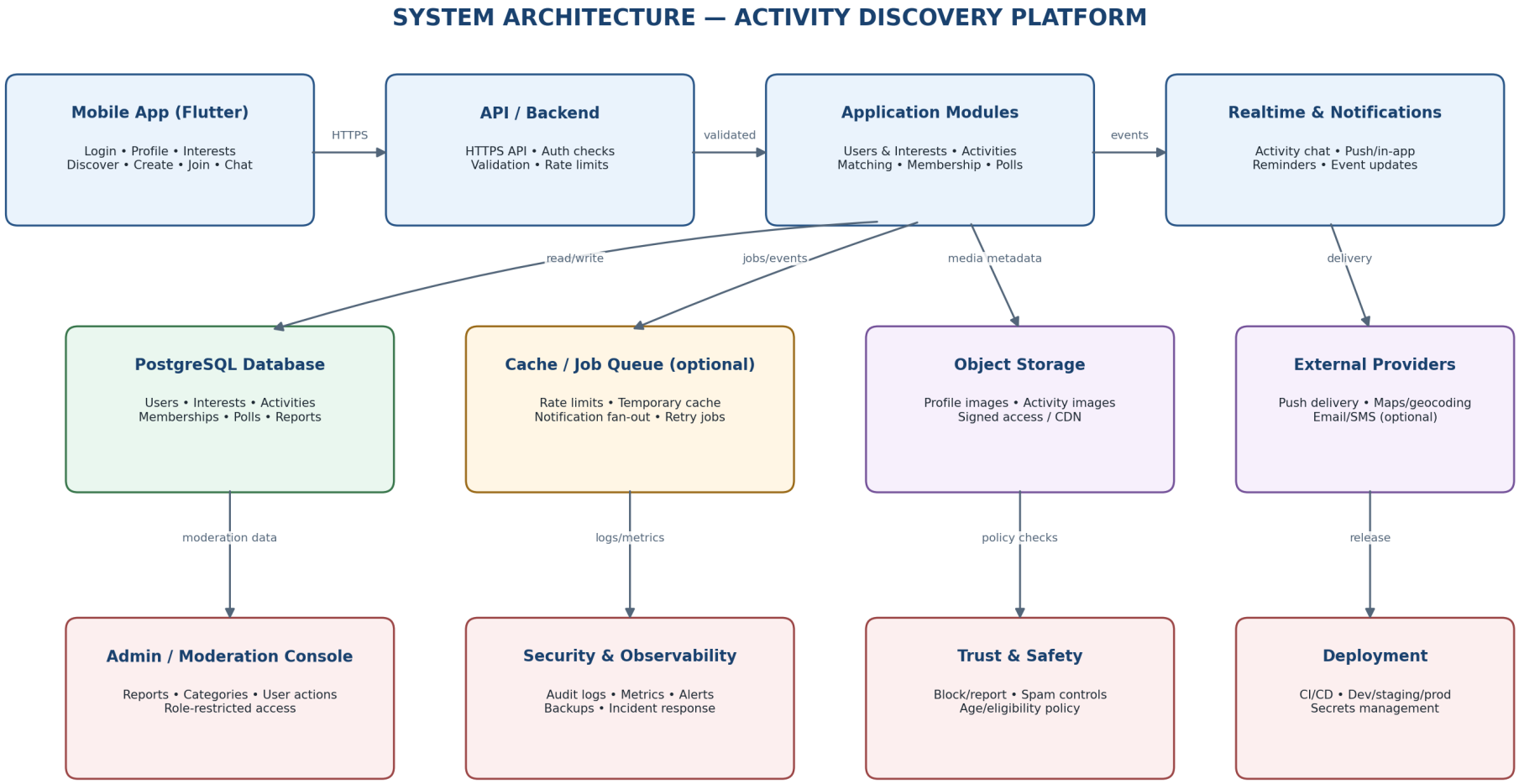
Activity Discovery, Matching & Group Coordination Platform

Version 1.0 | 12 October 2026

Architecture • Conceptual ERD • Activity workflow • Data design notes

1. System Architecture

The architecture separates the Flutter client from server-side business rules. The backend is authoritative for authorization, eligibility, activity lifecycle, join approvals, and participant capacity.



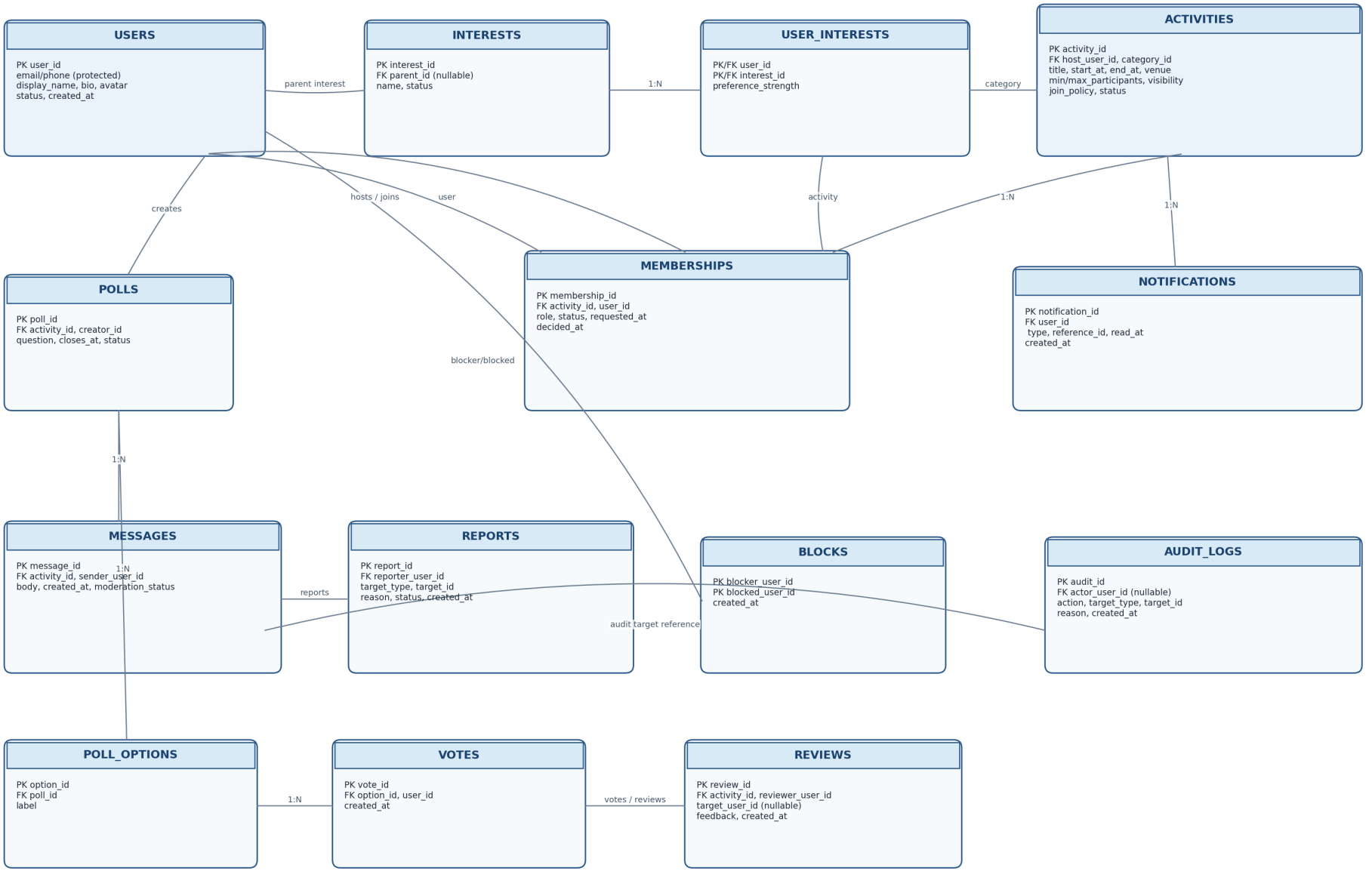
Core rule: authorization, eligibility, and capacity enforcement must happen on the server—not only in the app.

Recommended MVP stack: Flutter/Dart mobile client; versioned HTTPS API; PostgreSQL relational database; a realtime service for activity chat; push notifications; object storage for images; and a protected moderation/admin console. A cache or job queue can be added when load or retry needs justify it.

2. Conceptual Database ER Diagram

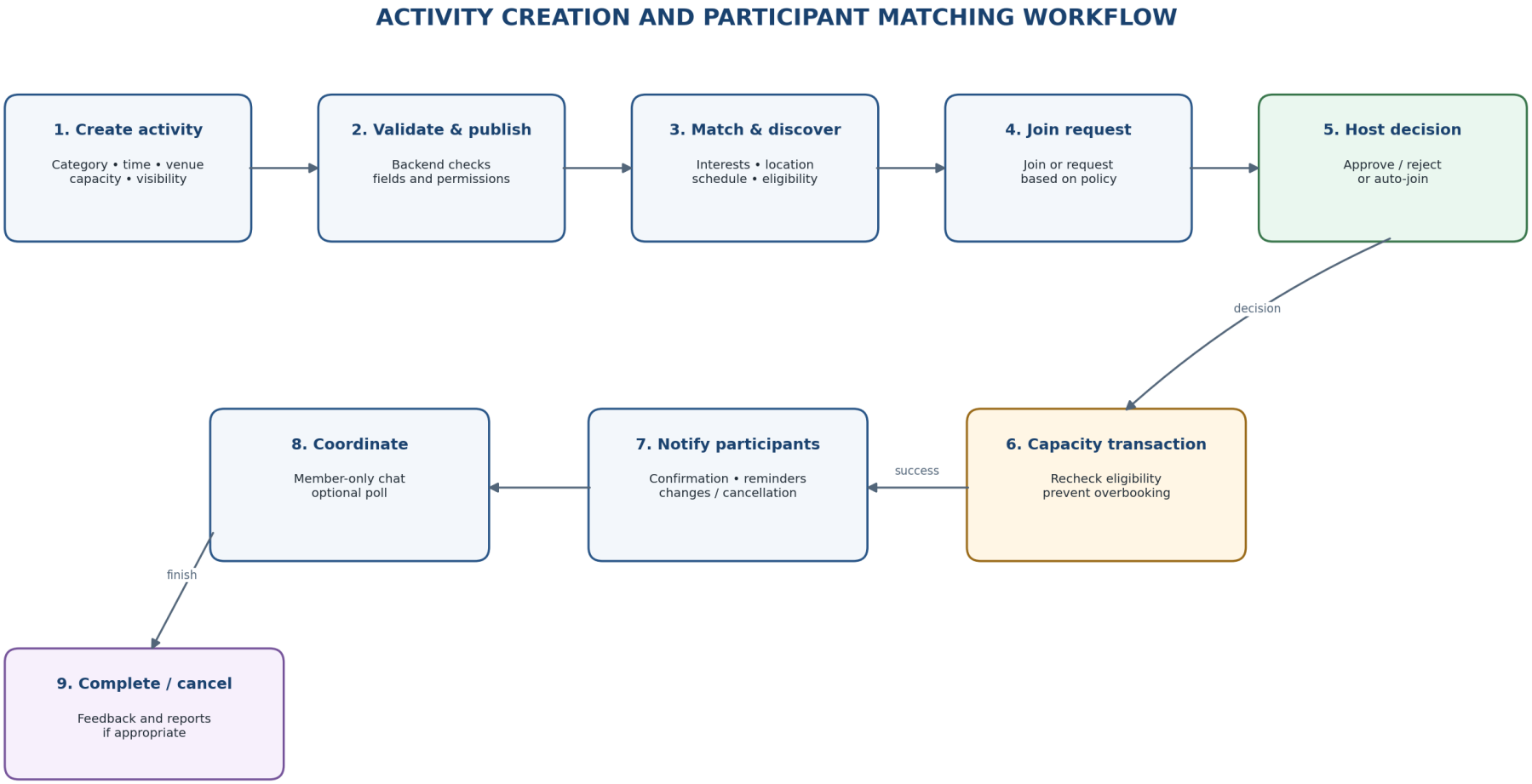
This diagram shows the major entities and relationships. It is a conceptual ERD; exact column types, indexes, constraints, and provider-specific auth fields should be finalized during implementation.

DATABASE ER DIAGRAM — CONCEPTUAL MODEL



PK = primary key • FK = foreign key • Composite keys shown where appropriate • Simplified conceptual ERD

3. Activity Creation and Matching Workflow



Important: users who share an interest should not automatically receive unlimited unsolicited messages. The system should make activities discoverable and send notifications only under user preferences, visibility rules, eligibility checks, and rate limits.

4. Core Database Tables

Table	Important columns / constraints	Purpose
users	user_id PK; auth_subject unique; display_name; bio; avatar_url; status; created_at	Account/profile. Keep credentials with trusted auth provider where possible.
interests	interest_id PK; parent_id FK interests; name; status	Hierarchical activity categories.
user_interests	user_id FK; interest_id FK; preference_strength; composite unique(user_id, interest_id)	Many-to-many user interests.
activities	activity_id PK; host_user_id FK users; category_id FK interests; start_at; end_at; venue; min_participants; max_participants; visibility; join_policy; status	Activity listing and lifecycle.
activity_memberships	membership_id PK; activity_id FK; user_id FK; role; status; requested_at; decided_at; unique(activity_id,user_id)	Host, pending request, accepted, rejected, waitlisted, left.
polls	poll_id PK; activity_id FK; creator_id FK users; question; closes_at; status	Optional group decision polls.
poll_options	option_id PK; poll_id FK polls; label	Options for each poll.
poll_votes	vote_id PK; option_id FK; user_id FK; created_at; unique(option_id,user_id)	Votes; apply poll-specific rules.
messages	message_id PK; activity_id FK; sender_user_id FK; body; created_at; moderation_status	Activity-scoped chat; enforce membership server-side.
notifications	notification_id PK; user_id FK; type; reference_id; read_at; created_at	In-app/push notification records.
blocks	blocker_user_id FK; blocked_user_id FK; created_at; composite PK(blocker_user_id,blocked_user_id)	User blocking relationship.
reports	report_id PK; reporter_user_id FK; target_type; target_id; reason; status; created_at	Safety reports; target may be polymorphic.
reviews	review_id PK; activity_id FK; reviewer_user_id FK; target_user_id nullable; feedback; created_at	Optional post-activity feedback.
admin_audit_logs	audit_id PK; actor_user_id nullable FK; action; target_type; target_id; reason; created_at	Audit of privileged actions.

5. Critical Integrity and Security Rules

- Use a unique constraint on (activity_id, user_id) so a user cannot occupy duplicate membership records.
- Use a database transaction or atomic conditional update to enforce max_participants under concurrent join requests.
- Store timestamps consistently (prefer UTC); display local time to each user.
- Enforce private activity, chat, and membership authorization on the backend for every request.
- Keep exact live location private by default; use a selected public venue or approximate area for discovery.
- Treat report targets carefully: validate target_type and target_id, and restrict report evidence to authorized moderators.

• Use pagination and indexes for activity discovery, especially category, status, start_at, and geographic filtering.

• Use soft-delete/anonymization and retention policies thoughtfully; document exceptions for abuse/security records.

• Use rate limits for registration, activity creation, invitations, join requests, and messages.

• Do not put passwords, access tokens, or private message content in application logs.

6. Suggested Indexes

Illustrative indexes: activities(status, start_at); activities(category_id, status, start_at); activity_memberships(activity_id, status); activity_memberships(user_id, status); messages(activity_id, created_at); notifications(user_id, read_at, created_at); reports(status, created_at); user_interests(interest_id, user_id). Add a spatial index if PostgreSQL/PostGIS is used for location queries.

7. Development order

1) Auth/profile/interests → 2) activity create/discover → 3) join request and atomic capacity → 4) notifications → 5) member-only chat → 6) block/report/admin moderation → 7) polls, reviews, recommendations. Keep the backend modular rather than starting with many microservices.